

React

PokeBall mapping - 2 pont

Módosítsa az src/ mappában található App.tsx komponenst úgy, hogy a usePokemonContext custom hook-ot használva kiválassza a pokemonsData értékét (ez a pokémonokat tartalmazó lista lesz), majd ezen végigmappelve PokeBall-okat (be kell importálni!) jelenít meg a kijelölt területen. A PokeBall-ok kulcs értéke az adott pokémon neve legyen.

App.tsx

```
const {selectedPokemon, nextPokemon, prevPokemon, pokemonsData} = usePokemonContext()
```

```
{/* TODO: itt kell mappelni a PokeBall-okat*/}  
{pokemonsData.map(pokemon => <PokeBall pokemon={pokemon} key={pokemon.name}/>)}
```

Adatok beolvasása API-ból - 4 pont

- Egészítse ki a src/context/ mappában lévő PokeContextProvider komponenst úgy, hogy a pokémonok adatai ne a dummyData alapján jelenjenek meg, hanem a pokemons.json fájlból beolvasva (fetch vagy axios, mivel a vizsgán az adatok backendtől fognak érkezni...) - 2p
- A sikeres lekérés után frissítse a pokeData state-t a kapott adatokkal, valamint állítsa be a selectedPokemon state-t a beolvasott tömb első elemére. - 2p

PokeContextProvider.tsx

```
// TODO: itt kell a json-ból olvasni
```

```
useEffect(()=>{  
  fetch("pokemons.json")
```



Szerkesztés a következővel: WPS Office

```
.then(response => response.json() )
.then(apiData => {
  setPokemonsData(apiData)
```

Health komponens - 9 pont

Készítsen egy újrahasznosítható React komponenst, amely egy pokémon életét jeleníti meg, 5 szív ikonnal!

- Hozza létre az src/components/Health.tsx fájlban a Health komponenst és exportálja! - 2p
- A komponens propsként kapjon meg egy számot, ami egy adott pokémon életét fogja jelezni a [0,5] intervallumon (lehet tört szám is!). - 1p
- A program által visszaadott elem egy div legyen, amelyet a health osztállyal lásson el. - 1p

Health.tsx

ES7 extension telepítése -> rafce -> react import kitörlése

```
type HealthPropsType = {
  hp: number
}

const Health = ({hp} : HealthPropsType) => {
  return (
    <div className="health">
      {hp}
    </div>
  )
}

export default Health
```

- Az src/ mappában található App komponensben használja fel az imént létrehozott Health komponenst a megjelölt helyen. A komponensnek átadott érték a selectedPokemon.health legyen, vagy ennek hiányában a 0 szám. - 2p

App.tsx

```
{/* TODO: ide jön a Health komponens */}
```



Szerkesztés a következővel: WPS Office

```
<Health hp={selectedPokemon.health} />
```

- Írja meg a program logikáját: A kimenet minden alkalommal pontosan 5 élet ikonból álljon, de az élet kitöltése a props-ként átadott számtól függjön. Az életek kirajzolásához használja a fent létrehozott konstansokat. - 3p

Health.tsx (az előző Health.tsx kód módosul)

```
const Health = ({hp} : HealthPropsType) => {  
  
  const healthCell = (idx: number) => {  
    if(Math.floor(hp) > idx) return fullHeart  
    if(hp > idx) return crackedHeart  
    return emptyHeart  
  }  
  
  return (  
    <div className="health">  
      {healthCell(0)}  
      {healthCell(1)}  
      {healthCell(2)}  
      {healthCell(3)}  
      {healthCell(4)}  
    </div>  
  )  
}
```

CSS feladatok

Az exam.css fájlba írja meg, hogy a .pokeBallWrapper osztályú elem egy grid szerkezetet vegyen fel, és 10px-es hézaggal 5 oszlopban jelenjen meg.

Ugyan ez az elem 600px-es ablakszélesség alatt 3 oszlopban jelenjen meg.

(NT elmondása szerint a vizsgában több CSS feladat lesz, de ilyen egészen biztosan)

exam.css (!!!véletlenül se nyúlj bele az index.css-be!!!)



Szerkesztés a következővel: WPS Office

```
/* TODO: CSS feladatok */

.pokeballWrapper{
  display: grid;
  grid-template-columns: repeat(5, 1fr);
  gap: 10px;
}

@media(max-width: 600px){
  .pokeballWrapper{
    grid-template-columns: repeat(3, 1fr);
  }
}
```

